

SuretySeven

SDE-1 Take-Home Assignment

Document Processing Pipeline

Time expectation: approximately 15–20 hours total over a maximum period of two weeks.

You are free to choose the programming language, backend/frontend framework, database, cloud/platform, and architecture. You may freely use AI tools such as ChatGPT, Claude, GitHub Copilot, Cursor, Kiro, etc.

We are interested in how you **design, build, test and reason about the system**, not whether you write every line manually.

Background

At SuretySeven, documents and data are received from brokers, insurers and other systems. A document may contain information that needs to be extracted and validated before it can be used by downstream systems.

For this assignment, build a simplified **Document Processing Service**. A user should be able to upload a document, have the system process it asynchronously, and inspect the extracted information and processing status.

You do **not** need to build a real OCR/AI system. The document-processing component can be mocked.

1. Upload a Document

Build an API that allows a user to upload a document.

POST /documents

The request should contain:

- File
- Document type
- Optional metadata

Support at least PDF. You may support additional formats if you wish.

```
{
  "documentId": "DOC-12345",
  "status": "UPLOADED"
}
```

2. Document Processing

After upload, the document should be processed asynchronously.

```
UPLOADED
↓
PROCESSING
↓
PROCESSED
```

or

```
PROCESSING
↓
FAILED
```

The actual processing can be mocked. For example, the mock processor could extract:

```
{
  "companyName": "ABC Construction Pvt Ltd",
  "registrationNumber": "U12345DL2020PTC123456",
  "address": "New Delhi",
  "annualRevenue": 12500000,
  "documentDate": "2026-08-15"
}
```

You may hard-code or simulate extraction based on the uploaded file.

3. Application Status

Provide an API such as:

```
GET /documents/{documentId}
```

It should return information such as:

```
{
  "documentId": "DOC-12345",
  "status": "PROCESSED",
  "documentType": "FINANCIAL_STATEMENT",
  "createdAt": "...",
  "updatedAt": "...",
  "result": {
    "companyName": "ABC Construction Pvt Ltd",
    "registrationNumber": "U12345DL2020PTC123456",
    "annualRevenue": 12500000
  }
}
```

The API should also return useful information when processing fails.

4. Validation

The extracted data should be validated. For example:

companyName	→ required
registrationNumber	→ required
annualRevenue	→ must be ≥ 0
documentDate	→ valid date

If extracted data is invalid, the document should not simply be marked as successfully processed. Decide how you want to represent validation failures.

5. Retry

Processing may fail. For example, the mock processor may randomly return:

```
SUCCESS
TIMEOUT
ERROR
INVALID_RESULT
```

Implement a retry mechanism. Decide:

- How many retries are allowed?
- Which errors should be retried?
- What happens after all retries fail?

You do not need to build a sophisticated distributed retry system. A simple, well-reasoned implementation is sufficient.

6. Duplicate Documents

Users may upload the same document multiple times. Consider how your system could identify duplicate uploads.

You may use any reasonable approach, such as a file hash, filename + size, content hash, or an explicit client-provided idempotency key. You do not need to implement every possibility. Explain your choice.

7. Processing History

A user should be able to understand what happened to a document.

```
GET /documents/{documentId}/history
```

For example:

```
[
  {"status": "UPLOADED", "timestamp": "..."},
  {"status": "PROCESSING", "timestamp": "..."},
  {"status": "FAILED", "timestamp": "...", "reason": "PROCESSOR_TIMEOUT"},
  {"status": "PROCESSING", "timestamp": "..."},
  {"status": "PROCESSED", "timestamp": "..."}
]
```

8. User Interface — Required

You must build a basic web UI for the system.

The purpose is to demonstrate that you can take your backend functionality and make it usable by a real user.

A. Document Upload

Provide a page/form where a user can:

- Select a document
- Select the document type
- Upload the document
- See whether the upload succeeded or failed

B. Document List

Provide a page showing previously uploaded documents. At minimum display:

- Document ID
- Filename
- Document type
- Status
- Upload date

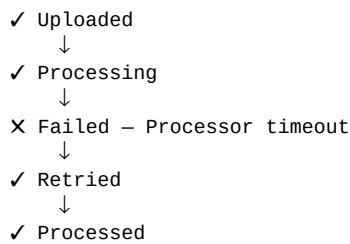
The list should support status filtering and document type filtering. Pagination is encouraged.

C. Document Details

Clicking a document should open a detail view showing:

- **Document information:** filename, document type, upload date, current status
- **Extracted information:** display extracted fields in a readable format

- **Processing history:** show the processing lifecycle in a user-friendly manner
- If validation fails, clearly display the validation errors.



9. UI Quality

The basic UI requirement is about **functionality and usability**, not graphic design.

- Clear layout
- Sensible labels
- Loading states
- Success/error states
- Empty states
- Reasonable responsiveness
- Clearly visible application/document status
- Do not expose raw backend errors directly to users

You may use React, Next.js, Vue, Angular, or plain HTML/CSS/JavaScript.

10. Advanced UI/UX — Bonus

Advanced UI/UX will receive bonus consideration.

You can earn bonus points by going beyond the basic requirements. Examples include:

- **Dashboard:** total documents, processing, processed and failed counts
- **Better filtering:** combinations of status, document type, date range and search
- **Processing experience:** progress indicators, automatic status refresh, retry indication and clear failure explanations
- **Document preview:** preview the uploaded PDF
- **Processing timeline:** polished visual timeline of processing attempts
- **Responsive design:** desktop and tablet/mobile support
- **UX polish:** skeleton loading, empty states, confirmation dialogs, toast notifications, keyboard accessibility and accessible form controls
- **Visual design:** a clean, professional interface that looks like an actual internal enterprise product rather than a basic developer demo

Do not sacrifice backend quality for UI polish. A beautiful UI with unreliable backend functionality will score lower than a simple UI backed by a well-engineered system.

11. Search and Filtering

Provide a way to find documents. At minimum support filtering by status and document type.

GET /documents?status=FAILED

Pagination is encouraged.

12. Observability

Provide enough logging to troubleshoot a failed document. A developer should be able to answer: *“Why did document DOC-12345 fail?”*

Include useful information such as documentId, processing attempt, status, failure reason and timestamp. Avoid logging sensitive document contents.

13. Testing

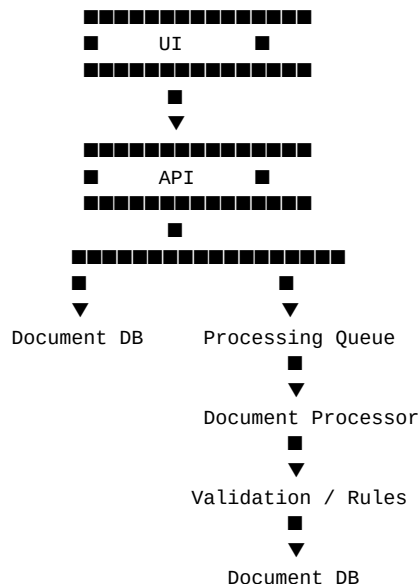
Write automated tests covering important functionality. At minimum include tests for:

- Valid document upload
- Invalid extracted data
- Successful processing
- Processor failure
- Failure followed by a successful retry
- Uploading the same document twice

You do not need 100% code coverage. We care about whether your tests provide meaningful confidence.

14. Architecture

Include a simple architecture diagram. The following is only an example:



This is only an example. **You should design your own architecture.** For this assignment, a simple monolithic application is completely acceptable.

15. Engineering Questions

- Why did you choose your architecture?
- Why did you choose your database?

- How does asynchronous processing work?
- How do retries work?
- How do you prevent duplicate processing?
- What happens if the application crashes during processing?
- What would you change if the system had to process 1 million documents per day?
- What are the biggest limitations of your implementation?

16. AI Usage

AI tools are explicitly allowed.

Include an **AI_USAGE.md** file describing:

- Which AI tools you used
- What you used them for
- Significant code/design generated by AI
- Any AI-generated output that you changed or rejected
- Anything you learned or discovered while reviewing AI output

You will be asked about this during the technical discussion.

Deliverables

```

README.md
AI_USAGE.md

/backend
/frontend

/tests

/docs
  architecture.png

```

The exact project structure is your choice. The project should be easy for another engineer to run, for example with `docker compose up` or equivalent instructions.

What we are looking for

- Understand a problem
- Break it into sensible components
- Build a working product
- Write maintainable code
- Handle errors
- Think about users
- Test their work
- Explain their decisions
- Use AI effectively without blindly trusting it

A simple, well-engineered solution is preferable to an unnecessarily complicated one.

A polished UI is a bonus, but **functionality and engineering quality come first.**